

Aula 2 - Javascript

Programação em Dispositivos Móveis

Carmen Silva

*Universidade Técnica do Atlântico
Instituto de Engenharia e Ciências do Mar*

26 de março de 2026

Programação

1 JavaScript

2 Destructuring

3 Promises, async e await

JavaScript

Spread e Rest Operator

- O **spread operator** e o **rest operator** usam a mesma sintaxe (...) em JavaScript, mas têm funções diferentes:
 - Spread operator → “espalha” os elementos de um array ou objeto, útil para copiar ou combinar dados.
 - Rest operator → “agrupa” múltiplos elementos ou propriedades em um array ou objeto, útil quando não sabemos quantos valores vamos receber.

Spread Operator em Arrays

- O spread operator permite expandir elementos de um array dentro de outro array:

```
1 const numeros1 = [1, 2, 3];  
2 const numeros2 = [4, 5, 6];  
3 const todos = [...numeros1, ...numeros2];  
4 console.log(todos); // [1, 2, 3, 4, 5, 6]
```

- Pode ser usado para copiar arrays sem modificar o original:

```
1 const original = [1, 2, 3];  
2 const copia = [...original];  
3 copia.push(4);  
4 console.log(copia); // [1, 2, 3, 4]
```

Spread Operator em Objetos

- Também é possível espalhar propriedades de objetos em novos objetos:

```
1 const obj1 = { a: 1, b: 2 };  
2 const obj2 = { c: 3, ...obj1 };  
3 console.log(obj2); // { c: 3, a: 1, b: 2 }
```

Rest Operator em Funções

- O rest operator permite agrupar múltiplos argumentos em um array:

```
1 function soma(...valores) {  
2     return valores.reduce((total, n) => total +  
3     n, 0);  
4 }  
4 console.log(soma(1, 2, 3, 4)); // 10
```

- **...valores** captura todos os argumentos restantes
- Permite funções com número variável de argumentos

Rest Operator em Arrays

- Com rest operator é possível desestruturar arrays e capturar o restante dos elementos:

```
1 const [primeiro, segundo, ...resto] = [1,2,3,4,5];  
2 console.log(primeiro); // 1  
3 console.log(resto);    // [3, 4, 5]
```

Rest Operator em Objetos

- O rest operator permite separar propriedades específicas e agrupar o restante:
 - Utilizando notação literal:

```
1 const pessoa = {nome:" Maria ", idade:60 , cidade  
  : " Mindelo "};  
2 const { nome, ...resto } = pessoa;  
3 console.log(nome); // "Maria"  
4 console.log(resto); // {idade: 60, cidade: "  
  Mindelo"}
```

Diferença Entre Spread e Rest

- Spread → usado na direita de = ou dentro de arrays/objetos para expandir elementos/propriedades
- Rest → usado na esquerda de = em desestruturação ou em parâmetros de função para agrupar múltiplos valores

```
1 // Spread
2 const a = [1, 2];
3 const b = [...a, 3]; // [1, 2, 3]
4
5 // Rest
6 const [x, ...resto] = [1, 2, 3]; // x=1, resto=[2, 3]
```

Destructuring

Destructuring

- O destructuring em JavaScript permite extrair valores de arrays ou propriedades de objetos e atribuí-los a variáveis de forma concisa.
- É muito útil para tornar o código mais limpo, reduzindo a necessidade de acessar manualmente cada elemento ou propriedade.

Destructuring de Arrays

- Com arrays, é possível extrair elementos usando parênteses retos []:

```
1 const numeros = [10, 20, 30];  
2 const [a, b, c] = numeros;  
3 console.log(a, b, c); // 10 20 30
```

- A ordem importa, o primeiro valor vai para a primeira variável, o segundo para a segunda, etc.

Pular Elementos em Arrays

- É possível ignorar elementos usando vírgulas:

```
1 const numeros = [1, 2, 3, 4];  
2 const [primeiro, , terceiro] = numeros;  
3 console.log(primeiro, terceiro); // 1 3
```

Valores Padrão

- Se o array não tiver elementos suficientes, é possível definir valores padrão:

```
1 const numeros = [5];  
2 const [a, b = 10] = numeros;  
3 console.log(a, b); // 5 10
```

Destructuring de Objetos

- Com objetos, utiliza-se chavetas {} e o nome das propriedades:

```
1 const pessoa = {nome: "Maria", idade: 60};  
2 const {nome, idade} = pessoa;  
3 console.log(nome, idade); // Maria 60
```

- A ordem não importa nos objetos, apenas os nomes das propriedades

Renomear Variáveis

- É possível atribuir propriedades a variáveis com nomes diferentes:

```
1 const pessoa = {nome: "Manuel", idade: 66};  
2 const {nome: n, idade: i} = pessoa;  
3 console.log(n, i); // Manuel 66
```

Destructuring com Rest Operator

- O rest operator pode ser combinado com destructuring para agrupar elementos ou propriedades restantes:

```
1 // Arrays
2 const numeros = [1, 2, 3, 4];
3 const [primeiro, ...resto] = numeros;
4 console.log(resto); // [2, 3, 4]
5
6 // Objetos
7 const pessoa = {nome: "Maria", idade: 60, cidade
8   : "Mindelo"};
9 const {nome, ...restoObj} = pessoa;
10 console.log(restoObj); // {idade: 60, cidade: "
    Mindelo"}
```

Promises, async e await

Código Assíncrono

- Em JavaScript, algumas operações demoram tempo (por exemplo: esperar 2 segundos, ler um ficheiro, fazer um cálculo pesado).
- Essas operações não bloqueiam o programa, ou seja, o código continua a executar.

```
1 console.log("A");  
2 setTimeout(() => {  
3     console.log("B");  
4 }, 2000);  
5 console.log("C");  
6 /**  
7  * A  
8  * B  
9  * C  
10 */
```

- Isto acontece porque `setTimeout` é **assíncrono** → Assíncrono “faz isto depois, enquanto o resto continua já”

Promise

- Uma **Promise** é um objeto que representa um valor que ainda não existe, mas que vai existir no futuro.
- É como uma “promessa”:
 - Pode dar certo
 - Pode dar errado

Criar uma Promise

■ Sintaxe base:

```
1 const promessa = new Promise((resolve, reject)
    => {
2     // c digo
3 });
```

- **new Promise(...)** → cria uma nova Promise
- **(resolve, reject) =>** → função executada automaticamente
- **resolve(valor)** → indica sucesso
- **reject(erro)** → indica erro

Promise: exemplo

```
1 const promessa = new Promise((resolve, reject) => {  
2     let sucesso = true;  
3  
4     if (sucesso) {  
5         resolve("Funcionou!");  
6     } else {  
7         reject("Erro!");  
8     }  
9 }) ;
```

Utilizar uma Promise (then e catch)

```
1 promessa
2   .then(resultado => {
3       console.log("Sucesso:", resultado);
4   })
5   .catch(erro => {
6       console.log("Erro:", erro);
7   });
```

- **.then(...)** → executa quando resolve() é chamado
- **.catch(...)** → executa quando reject() é chamado
- **resultado** → valor passado no resolve()
- **erro** → valor passado no reject()

Promise: exemplo

```
1 return new Promise(resolve => setTimeout(() => resolve(Terminei!); ,  
    2000); );  
2 esperar2s().then(msg => console.log(msg));
```

- **return new Promise(...)** → devolve uma Promise
- **setTimeout(...)** → espera 2 segundos
- **resolve("Terminei!")** → termina a Promise

async

- **async** serve para declarar uma função assíncrona.

```
1 async function exemplo() {  
2     return "OI ";  
3 }
```

- Esta função sempre retorna uma Promise.

- Equivale a:

```
1 function exemplo() {  
2     return Promise.resolve("OI ");  
3 }
```

await

- **await** serve para esperar o resultado de uma Promise.
- Só pode ser usado dentro de funções **async**.

```
1 function esperar2s() {  
2     return new Promise(resolve => {  
3         setTimeout(() => {  
4             resolve("Terminei!");  
5         }, 2000);  
6     });  
7 }  
8  
9 async function executar() {  
10     const resultado = await esperar2s();  
11     console.log(resultado);  
12 }  
13  
14 executar();
```

- **await esperar2s()** → pausa a função até a Promise terminar
- **resultado** → valor do resolve()

Promise vs async/await

■ Com **.then()**

```
1 esperar2s () .then ( res => console.log ( res ) ) ;
```

■ Com **await**

```
1 const res = await esperar2s () ;  
2 console.log ( res ) ;
```

■ async/await é mais fácil de ler.

Tratar erros com try/catch

```
1 async function exemplo() {  
2     try {  
3         const res = await esperar2s();  
4         console.log(res);  
5     } catch (erro) {  
6         console.log("Erro:", erro);  
7     }  
8 }
```

- **try** → tenta executar
- **catch** → captura erros (reject)

Várias Promises

```
1 async function exemplo() {  
2     const a = await esperar2s();  
3     const b = await esperar2s();  
4     console.log(a, b);  
5 }
```

- Executa uma depois da outra.

Várias Promises

```
1 async function exemplo() {  
2     const [a, b] = await Promise.all([esperar2s(),  
    esperar2s()]);  
3     console.log(a, b);  
4 }
```

- **Promise.all([...])** → executa tudo em paralelo
- Retorna um array com os resultados